TD2 — Industrialiser K3s avec Terraform, Ansible et Helm

Du geste manuel au pipeline IaC déclaratif

Cours : IaC / DevSecOps | **Niveau :** 5e année — Architecture logicielle

Durée indicative : **3 h** | Environnement : DevStack OpenStack — projet `asl-project`

Version 2 — corrigée après validation en environnement réel

Objectifs pédagogiques

À l'issue de ce TD, vous serez capable de **reproduire intégralement l'infrastructure du TD1** (VM K3s opérationnelle exposant une application) non plus en tapant des commandes, mais en **décrivant l'état cible** dans trois langages complémentaires :

— **Terraform** pour *provisionner* les ressources OpenStack (VM, réseau, sécurité, IP flottante) ;

— **Ansible** pour *configurer* la VM et y installer K3s ;

— **Helm** pour *déployer* une application 3-tiers (frontend, backend, base de données) dans le cluster.

À la fin du TD, vous aurez déployé une application **NovaShop** accessible via `http://<IP-FLOTTANTE>`, avec une interface web montrant en temps réel les pods qui servent chaque requête. C'est **le cœur du métier DevSecOps** : passer de l'artisanat à l'industrialisation.

Compétences visées (référentiel 5A)

À l'issue du TD, l'étudiant est capable de :

— **C1** — Écrire un projet Terraform complet (provider, variables, data sources, resources, outputs).

— **C2** — Utiliser les `data sources` pour référencer une infrastructure existante sans la recréer.

— **C3** — Écrire un inventaire Ansible et des rôles idempotents.

— **C4** — Distinguer *provisionnement* (Terraform), *configuration* (Ansible) et *déploiement applicatif* (Helm).

— **C5** — Concevoir et déployer un ensemble de charts Helm pour une application 3-tiers.

— **C6** — Gérer la persistance avec un `StatefulSet` et un `PersistentVolumeClaim`.

— **C7** — Valider la *reproductibilité* et la *destructibilité* de l'infrastructure.

— **C8** — Utiliser un tunnel SSH pour piloter une API Kubernetes sans exposer ses ports sensibles.

Rappel

Note de version. Cette version 2 intègre les corrections identifiées lors de la validation du TD sur une infrastructure OpenStack réelle :

— usage de `openstack_networking_floatingip_associate_v2` (l'ancienne ressource Compute étant dépréciée) ;
— ajout de `PGDATA` dans le StatefulSet Postgres ;
— ajout du header HTTP `X-Pod-Name` pour prouver visuellement le load-balancing ;
— ajout d'une section dédiée au *tunnel SSH* vers l'API K3s ;
— interface HTML enrichie (cards + tableau + auto-refresh).

Légende des contextes d'exécution

Rappel

Dans ce TD, la majorité des commandes s'exécutent sur **l'hôte DevStack** (le « poste IaC »). Une seule machine distante entre en jeu : la VM K3s. Les bandeaux de couleur vous rappellent *où* vous tapez.

Bandeau	Machine	Rôle
Bleu	**Hôte DevStack** (`stack@devstack`)	Poste IaC : Terraform, Ansible, Helm, kubectl.
Orange	**VM K3s** (`ubuntu@k3s-<login>`)	Configurée automatiquement par Ansible.

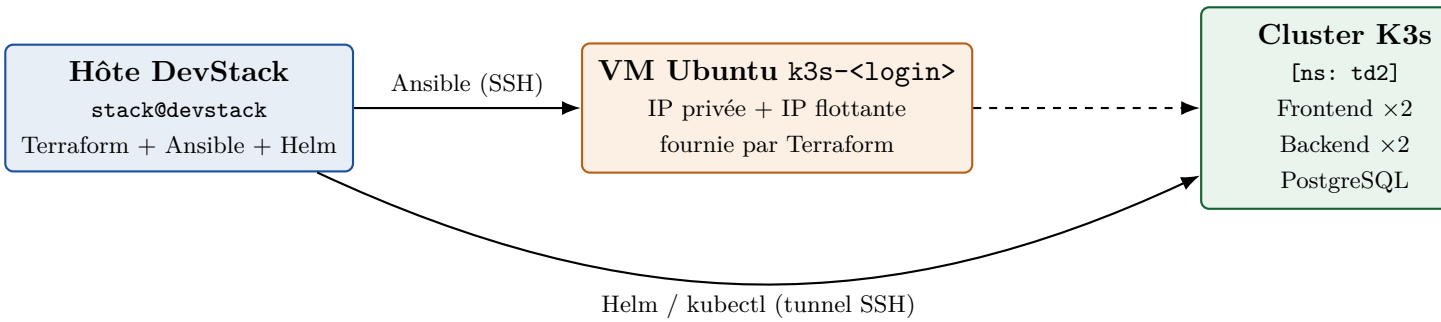
Astuce

Règle d'or. *Bleu = je décris l'infra. Orange = je vérifie de temps en temps.* En TD2, vous ne taperez presque **jamais** de commandes à la main dans la VM : c'est Ansible qui s'en charge. Le SSH manuel ne sert qu'à *vérifier* ce qu'Ansible a fait.

Contexte et architecture

Rappel

Fil rouge. Dans le TD1, vous avez monté à la main un cluster K3s exposant une application `hello` sur 2 réplicas. L'équipe *NovaShop* a apprécié, mais elle veut aller plus loin : elle veut **un pipeline reproductible** qui déploie une **vraie application 3-tiers** (frontend, backend, base de données) en une seule commande, dans un environnement recréable à volonté. Vous êtes chargé de l'industrialisation.
Hypothèse de travail. Le socle OpenStack du TD1 est *déjà en place* : projet `asl-project`, utilisateur `asl-user`, réseau privé `asl-net`, routeur `asl-router`, paire de clés `asl-key`, image `ubuntu-22.04-asl`. Terraform les **référencera** via des `data sources` — il ne les recréera pas.



Définition

Infrastructure as Code (IaC). Pratique qui consiste à décrire une infrastructure (machines, réseau, sécurité, applications) dans des **fichiers texte versionnés**, plutôt qu'à la construire à la main. Trois propriétés fondamentales :

— **Déclaratif** : on décrit l'*état cible*, pas les étapes.
— **Idempotent** : relancer produit le même résultat.
— **Versionnable** : tout est dans Git, revu en PR, audité.

Règles du TD

— Remplacez `<login>` par votre identifiant, en minuscules (ex. `alice`).
— Ne committez **jamais** `terraform.tfstate`, `*.tfvars` contenant des secrets, ni `kubeconfig`. Le `.gitignore` doit les exclure.
— Le mot de passe Postgres en clair dans un `values.yaml` est **assumé** ici (labo isolé). Nous verrons en TD3 comment le protéger.
— Conservez la sortie de chaque commande de validation.
— **Respectez les bandeaux** : Terraform, Ansible, Helm s'exécutent sur DevStack. La VM K3s ne se visite qu'en SSH pour vérifier.

Étape 0 — Préparer le poste IaC (DevStack)

0.1 — Se connecter et se placer en `stack`

Votre poste de travail | `utilisateur local`

```
ssh devstack
sudo su - stack
```

0.2 — Charger les identifiants ASL

Hôte DevStack | `stack@devstack`

```
source ~/devstack/asl-openrc
openstack token issue
```

Critère de validation

Le projet retourné doit être `asl-project`. Sinon, la commande `source` a échoué.

0.3 — Installer Terraform, Ansible, Helm, kubectl

Hôte DevStack | stack@devstack

```
# Terraform (HashiCorp)
sudo apt-get update -y
sudo apt-get install -y gnupg software-properties-common curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | \
  sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
  sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt-get update -y
sudo apt-get install -y terraform

# Ansible
sudo apt-get install -y ansible sshpass

# kubectl (binaire statique)
curl -LO "https://dl.k8s.io/release/$(curl -sL https://dl.k8s.io/release/stable.txt)/bin/
    linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
rm -f kubectl

# Helm
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

0.4 — Vérifier les versions

Hôte DevStack | stack@devstack

```
terraform version
ansible --version | head -1
helm version --short
kubectl version --client
```

Critère de validation

Les 4 outils répondent. Versions attendues : `terraform >= 1.6`, `ansible-core >= 2.15`, `helm 3.x`, `kubectl 1.30+`.

0.5 — Créer l'arborescence du projet

Hôte DevStack | stack@devstack

```
mkdir -p ~/td2/{terraform,ansible,helm}
cd ~/td2
ls -la
```

Rappel

Un projet IaC s'organise en dossiers d'outils. Terraform, Ansible et Helm n'ont pas les mêmes conventions : les mélanger dans un dossier plat est une source d'erreurs. On sépare physiquement les trois langages.

Étape 1 — Initialiser le projet Terraform et le provider OpenStack

1.1 — Le fichier `providers.tf`

Hôte DevStack | `stack@devstack`

```
cd ~/td2/terraform
cat > providers.tf <<'EOF'
# On declare les providers necessaires et on verrouille les versions.
terraform {
  required_version = ">= 1.6"
  required_providers {
    openstack = {
      source = "terraform-provider-openstack/openstack"
      version = "~> 1.54"
    }
  }
}

provider "openstack" {}
EOF
```

Terraform — Provisionner

Terraform ne parle jamais à l'API OpenStack directement : il passe par le *provider* `openstack`, un binaire téléchargé par `terraform init` qui traduit le HCL en appels REST Keystone/Nova/Neutron/Glance.

1.2 — Variables et valeurs

Hôte DevStack | `stack@devstack`

```
cat > variables.tf <<'EOF'
variable "login" {
  description = "Identifiant etudiant (minuscules)."
  type = string
  validation {
    condition = can(regex("^[a-z0-9]+$", var.login))
    error_message = "Le login doit contenir uniquement des minuscules et chiffres."
  }
}

variable "image_name" {
  type = string
  default = "ubuntu-22.04-asl"
}

variable "flavor_name" {
  type = string
  default = "m1.medium"
}

variable "keypair_name" {
  type = string
  default = "asl-key"
}

variable "private_network" {
  type = string
  default = "asl-net"
```

```
}

variable "public_network" {
  type = string
  default = "public"
}
EOF

cat > terraform.tfvars <<'EOF'
# /!\ Fichier dans .gitignore : peut contenir des secrets.
login = "alice" # <-- a remplacer par votre login
EOF
```

1.3 — `terraform init`

Hôte DevStack | `stack@devstack`

```
terraform init
```

Critère de validation

Sortie : `Terraform has been successfully initialized!` Le dossier `.terraform/` et le fichier `.terraform.lock.hcl` sont créés.

1.4 — `.gitignore`

Hôte DevStack | `stack@devstack`

```
cd ~/td2
cat > .gitignore <<'EOF'
# Terraform
*.tfstate
*.tfstate.*
.terraform/
crash.log
*.tfvars
!example.tfvars

# Ansible
*.retry
inventory.ini

# Kubernetes
kubeconfig
*.kubeconfig
EOF
```

Point DevSecOps

Pourquoi ignorer `*.tfstate` ? Le state Terraform contient *tout* : identifiants, mots de passe, contenus de `user_data`. En équipe, on le stocke dans un **backend distant chiffré** (S3 + KMS, Swift, Terraform Cloud). Ici, il reste local.

Étape 2 — Déclarer les ressources partagées via `data sources`

Définition

Un **data source** interroge un provider pour *lire* des informations sur une ressource *existante*, sans la gérer. Terraform peut la référencer mais ne la créera ni ne la supprimera jamais.

2.1 — Les data sources

Hôte DevStack | `stack@devstack`

```
cd ~/td2/terraform
cat > data.tf <<'EOF'
# Recuperer les ressources existantes (posees en TD1).

data "openstack_images_image_v2" "ubuntu" {
  name = var.image_name
  most_recent = true
}

data "openstack_compute_flavor_v2" "k3s" {
  name = var.flavor_name
}

data "openstack_networking_network_v2" "asl_net" {
  name = var.private_network
}

data "openstack_networking_subnet_v2" "asl_subnet" {
  name = "asl-subnet"
  network_id = data.openstack_networking_network_v2.asl_net.id
}

data "openstack_networking_network_v2" "public" {
  name = var.public_network
}

data "openstack_compute_keypair_v2" "asl_key" {
  name = var.keypair_name
}
EOF

terraform validate
terraform plan
```

Critère de validation

`terraform plan` doit afficher : `No changes. Your infrastructure matches the configuration.` Cela signifie que Terraform a bien lu les ressources existantes sans rien créer.

Terraform — Provisionner

`data` vs `resource` : la distinction fondamentale.

— `resource "X" "y"` : *je crée et je possède.* `destroy` le supprimera.

— `data "X" "y"` : *je lis, je ne possède pas.* `destroy` n'y touche pas.

En entreprise, cette nuance est **vitale** : elle évite qu'un `terraform destroy` d'applicatif efface

le réseau de toute l'entreprise.

Étape 3 — Créer le Security Group via Terraform

3.1 — Le fichier `security.tf`

Hôte DevStack | `stack@devstack`

```
cd ~/td2/terraform
cat > security.tf <<'EOF'
resource "openstack_networking_secgroup_v2" "k3s" {
  name = "k3s-${var.login}-sg"
  description = "TD2 K3s ${var.login} : SSH + HTTP + HTTPS"
}

resource "openstack_networking_secgroup_rule_v2" "ssh" {
  direction = "ingress"
  ethertype = "IPv4"
  protocol = "tcp"
  port_range_min = 22
  port_range_max = 22
  remote_ip_prefix = "0.0.0.0/0"
  security_group_id = openstack_networking_secgroup_v2.k3s.id
}

resource "openstack_networking_secgroup_rule_v2" "http" {
  direction = "ingress"
  ethertype = "IPv4"
  protocol = "tcp"
  port_range_min = 80
  port_range_max = 80
  remote_ip_prefix = "0.0.0.0/0"
  security_group_id = openstack_networking_secgroup_v2.k3s.id
}

resource "openstack_networking_secgroup_rule_v2" "https" {
  direction = "ingress"
  ethertype = "IPv4"
  protocol = "tcp"
  port_range_min = 443
  port_range_max = 443
  remote_ip_prefix = "0.0.0.0/0"
  security_group_id = openstack_networking_secgroup_v2.k3s.id
}
EOF

terraform plan
terraform apply
# yes
```

Critère de validation

`Apply complete! Resources: 4 added.` Vérification : `openstack security group show k3s-alice-sg`

Point DevSecOps

Le SG n'ouvre **que** 22, 80, 443. Pas de 6443 (API K3s), pas de 8472 (VXLAN), pas de 10250 (kubelet). Ces ports ne sont nécessaires qu'en multi-nœuds. **Principe de moindre privilège réseau**.

Étape 4 — Créer la VM et l'IP flottante via Terraform

4.1 — Le cloud-init

Rappel

Le `cloud-init` reste identique à TD1 : création de l'utilisateur `ubuntu` avec mot de passe `azerty`. Terraform l'injecte via l'attribut `user_data`.

Hôte DevStack | `stack@devstack`

```
cd ~/td2/terraform
cat > cloud-init.yaml <<'EOF'
#cloud-config
ssh_pwauth: true
chpasswd:
  expire: false
  users:
    - name: ubuntu
      password: azerty
      type: text
EOF
```

4.2 — Le fichier `compute.tf`

Piège classique

Dépréciation 2024. La ressource `openstack_compute_floatingip_associate_v2` est **dépréciée** au profit de `openstack_networking_floatingip_associate_v2` (côté Neutron). L'ancienne fonctionne encore mais disparaîtra. On utilise la nouvelle, avec un `data source` pour récupérer le port réseau de la VM.

Hôte DevStack | `stack@devstack`

```
cat > compute.tf <<'EOF'
# --- VM K3s ---
resource "openstack_compute_instance_v2" "k3s" {
  name = "k3s-${var.login}"
  image_id = data.openstack_images_image_v2.ubuntu.id
  flavor_id = data.openstack_compute_flavor_v2.k3s.id
  key_pair = data.openstack_compute_keypair_v2.asl_key.name
  security_groups = [openstack_networking_secgroup_v2.k3s.name]

  network {
    name = data.openstack_networking_network_v2.asl_net.name
  }

  user_data = file("${path.module}/cloud-init.yaml")

  metadata = {
    owner = var.login
```

```
    project = "asl"
    td = "td2"
  }
}

# --- IP flottante ---
resource "openstack_networking_floatingip_v2" "k3s_fip" {
  pool = data.openstack_networking_network_v2.public.name
}

# --- Recuperer le port reseau de la VM (necessaire cote Neutron) ---
data "openstack_networking_port_v2" "k3s_port" {
  device_id = openstack_compute_instance_v2.k3s.id
  network_id = data.openstack_networking_network_v2.asl_net.id
}

# --- Association FIP <-> port VM (API Neutron) ---
resource "openstack_networking_floatingip_associate_v2" "k3s_fip_assoc" {
  floating_ip = openstack_networking_floatingip_v2.k3s_fip.address
  port_id = data.openstack_networking_port_v2.k3s_port.id
}
EOF

terraform fmt
terraform validate
terraform plan
terraform apply
# yes
```

Critère de validation

`Apply complete! Resources: 3 added.` La VM est `ACTIVE` : `openstack server show k3s-alice -c status -c addresses`

Piège classique

Si `No valid host was found` : ressources insuffisantes sur les compute nodes. Vérifiez avec `openstack hypervisor stats show`. Si nécessaire, arrêtez des VMs de test ou choisissez un flavor plus petit.

Étape 5 — Exporter les sorties Terraform

Hôte DevStack | `stack@devstack`

```
cd ~/td2/terraform
cat > outputs.tf <<'EOF'
output "vm_name" { value = openstack_compute_instance_v2.k3s.name }
output "floating_ip" { value = openstack_networking_floatingip_v2.k3s_fip.address }
output "private_ip" { value = openstack_compute_instance_v2.k3s.access_ip_v4 }
output "ssh_command" { value = "ssh ubuntu@${openstack_networking_floatingip_v2.k3s_fip.
    address}" }
EOF

terraform apply -refresh-only
# yes
terraform output
```

Critère de validation

Les 4 outputs s'affichent. `terraform output -raw floating_ip` renvoie une IP seule.

Astuce

Chaque fois que vous ajoutez un output, lancez `terraform apply -refresh-only`. Cette commande met à jour le state sans rien créer ni détruire.

5.1 — Test SSH

Hôte DevStack | `stack@devstack`

```
export FIP=$(terraform output -raw floating_ip)
echo "IP flottante : $FIP"

until nc -z "$FIP" 22; do echo "En attente SSH..."; sleep 5; done

sshpass -p azerty ssh -o StrictHostKeyChecking=no ubuntu@"$FIP" \
  'hostname; echo "---"; ip -o -4 addr show ens3'
```

Critère de validation

Le hostname est `k3s-<login>`, l'IP privée est dans le range du subnet ASL.

Étape 6 — Construire l'inventaire Ansible

Ansible — Configurer

Ansible est un outil d'automatisation *agentless* : il se connecte en SSH et exécute des tâches idempotentes. Pas de daemon à installer sur les cibles.

6.1 — Générer l'inventaire depuis Terraform

Hôte DevStack | `stack@devstack`

```
cd ~/td2/terraform
cat >> outputs.tf <<'EOF'

output "ansible_inventory" {
  value = <<-EOT
    [k3s]
    k3s-${var.login} ansible_host=${openstack_networking_floatingip_v2.k3s_fip.address}

    [k3s:vars]
    ansible_user=ubuntu
    ansible_password=azerty
    ansible_ssh_common_args='-o StrictHostKeyChecking=no'
    ansible_python_interpreter=/usr/bin/python3
  EOT
}
EOF

terraform apply -refresh-only
# yes
terraform output -raw ansible_inventory > ~/td2/ansible/inventory.ini
cat ~/td2/ansible/inventory.ini
```

6.2 — Configurer Ansible

Hôte DevStack | stack@devstack

```
cd ~/td2/ansible
cat > ansible.cfg <<'EOF'
[defaults]
inventory = inventory.ini
host_key_checking = False
retry_files_enabled = False
stdout_callback = yaml
interpreter_python = auto_silent
deprecation_warnings = False
EOF

ansible -m ping all
```

Critère de validation

Sortie attendue : `"ping": "pong"` pour `k3s-alice`. Sinon : vérifiez le mot de passe, le SG, l'IP.

Étape 7 — Rôle Ansible k3s-prereq

Hôte DevStack | stack@devstack

```
mkdir -p ~/td2/ansible/roles/{k3s-prereq,k3s,kubeconfig}/tasks

cat > ~/td2/ansible/roles/k3s-prereq/tasks/main.yml <<'EOF'
---
# Preparation du systeme avant installation de K3s.
# Toutes les taches sont idempotentes.

- name: Desactiver systemd-resolved (reverrouille /etc/resolv.conf)
  ansible.builtin.systemd:
    name: systemd-resolved
    state: stopped
    enabled: false
  become: true
  failed_when: false

- name: Ecrire un /etc/resolv.conf statique
  ansible.builtin.copy:
    dest: /etc/resolv.conf
    content: |
      nameserver 1.1.1.1
      nameserver 8.8.8.8
    mode: '0644'
  become: true

- name: Ajouter l'entree locale dans /etc/hosts
  ansible.builtin.lineinfile:
    path: /etc/hosts
    line: "127.0.1.1 {{ inventory_hostname }}"
    state: present
  become: true

- name: Mettre a jour le cache apt
  ansible.builtin.apt:
    update_cache: true
```

```
    cache_valid_time: 3600
  become: true

- name: Installer les paquets prerequis
  ansible.builtin.apt:
    name:
      - curl
      - ca-certificates
      - netcat-openbsd
    state: present
  become: true

- name: Verifier que get.k3s.io est resolvable
  ansible.builtin.command: getent ahostsv4 get.k3s.io
  register: dns_check
  changed_when: false
  failed_when: dns_check.rc != 0
EOF

cat > ~/td2/ansible/playbook.yml <<'EOF'
---
- name: Configurer la VM K3s
  hosts: k3s
  gather_facts: true

  roles:
    - k3s-prereq
EOF

cd ~/td2/ansible
ansible-playbook playbook.yml
```

Critère de validation

Premier passage : `changed=4` (ou plus). Deuxième passage : `changed=0`. **Test d'idempotence obligatoire.**

Étape 8 — Rôle Ansible k3s

Hôte DevStack | `stack@devstack`

```
cat > ~/td2/ansible/roles/k3s/tasks/main.yml <<'EOF'
---
- name: Creer le dossier de configuration K3s
  ansible.builtin.file:
    path: /etc/rancher/k3s
    state: directory
    mode: '0755'
  become: true

- name: Ecrire la configuration K3s
  ansible.builtin.copy:
    dest: /etc/rancher/k3s/config.yaml
    content: |
      node-name: {{ inventory_hostname }}
      write-kubeconfig-mode: "0640"
      write-kubeconfig-group: "ubuntu"
    mode: '0644'
  become: true
```

```
- name: Verifier si K3s est deja installe
  ansible.builtin.stat:
    path: /usr/local/bin/k3s
  register: k3s_binary

- name: Installer K3s
  ansible.builtin.shell: |
    curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="server" sh -
  args:
    creates: /usr/local/bin/k3s
  when: not k3s_binary.stat.exists
  become: true

- name: Activer et demarrer le service K3s
  ansible.builtin.systemd:
    name: k3s
    state: started
    enabled: true
  become: true

- name: Attendre que l'API reponde sur 6443
  ansible.builtin.wait_for:
    port: 6443
    host: 127.0.0.1
    delay: 3
    timeout: 120
  become: true

- name: Attendre que le noeud soit Ready
  ansible.builtin.shell: |
    /usr/local/bin/k3s kubectl wait \
      --for=condition=Ready node/{{ inventory_hostname }} --timeout=180s
  register: node_ready
  changed_when: false
  retries: 3
  delay: 10
  until: node_ready.rc == 0
  become: true
EOF

cat > ~/td2/ansible/playbook.yml <<'EOF'
---
- name: Configurer la VM K3s
  hosts: k3s
  gather_facts: true
  roles:
    - k3s-prereq
    - k3s
EOF

cd ~/td2/ansible
ansible-playbook playbook.yml
```

Critère de validation

Le nœud est `Ready`. Vérification :

```
sshpass -p azerty ssh -o StrictHostKeyChecking=no ubuntu@"$FIP" \
  'kubectl get nodes -o wide'
```

Étape 9 — Récupérer le kubeconfig sur DevStack

9.1 — Créer le rôle kubeconfig

Piège classique

Piège du remplacement d'IP. Utilisez `replace()` (remplacement littéral) et *non* `regex_replace()` : les backslashes sont consommés par YAML puis Jinja, rendant la regex fragile.

Hôte DevStack | stack@devstack

```
cat > ~/td2/ansible/roles/kubeconfig/tasks/main.yml <<'EOF'
---
- name: Lire le kubeconfig depuis la VM
  ansible.builtin.slurp:
    src: /etc/rancher/k3s/k3s.yaml
  register: kubeconfig_raw
  become: true

- name: Reecrire l'adresse du serveur API
  ansible.builtin.set_fact:
    kubeconfig_fixed: "{{ kubeconfig_raw.content | b64decode | replace('127.0.0.1',
        ansible_host) }}"

- name: Ecrire le kubeconfig sur DevStack
  ansible.builtin.copy:
    dest: "{{ lookup('env', 'HOME') }}/td2/ansible/kubeconfig"
    content: "{{ kubeconfig_fixed }}"
    mode: '0600'
  delegate_to: localhost
  become: false
EOF

cat > ~/td2/ansible/playbook.yml <<'EOF'
---
- name: Configurer la VM K3s
  hosts: k3s
  gather_facts: true
  roles:
    - k3s-prereq
    - k3s
    - kubeconfig
EOF

ansible-playbook playbook.yml
grep server ~/td2/ansible/kubeconfig
```

Critère de validation

Le kubeconfig contient `server: https://<FIP>:6443`. Le fichier est en mode `0600`.

Étape 9.bis — Tunnel SSH vers l'API K3s

Définition

Un **tunnel SSH** est un canal chiffré qui redirige un port local vers un port distant via SSH. On l'utilise pour joindre une API interne sans l'exposer sur Internet.

Rappel

Pourquoi un tunnel ? Le SG n'ouvre que 22, 80, 443. Le port 6443 (API K3s) est volontairement fermé. Or `kubectl` et `helm` ont besoin de joindre cette API depuis DevStack. Solution : un tunnel SSH local qui redirige `localhost:6443` vers le port 6443 de la VM à travers la connexion SSH déjà autorisée.

9.bis.1 — Lancer le tunnel

Hôte DevStack | `stack@devstack`

```
cd ~/td2/terraform
export FIP=$(terraform output -raw floating_ip)

# Verifier qu'aucun tunnel n'ecoute deja
ss -tlnp | grep :6443 || echo "port libre"

# Lancer le tunnel en arriere-plan
sshpass -p azerty ssh -f -N \
  -o StrictHostKeyChecking=no \
  -o ExitOnForwardFailure=yes \
  -L 6443:127.0.0.1:6443 \
  ubuntu@"$FIP"

sleep 1
ss -tlnp | grep :6443
```

Critère de validation

Une ligne `LISTEN ... 127.0.0.1:6443` apparaît. Le tunnel est actif.

9.bis.2 — Adapter le kubeconfig et tester

Hôte DevStack | `stack@devstack`

```
# Le kubeconfig doit pointer sur 127.0.0.1 (pas la FIP)
sed -i "s|server: https://$FIP:6443|server: https://127.0.0.1:6443|" \
  ~/td2/ansible/kubeconfig
grep server ~/td2/ansible/kubeconfig

export KUBECONFIG=$HOME/td2/ansible/kubeconfig
kubectl get nodes -o wide
kubectl get pods -A
```

Critère de validation

Le nœud apparaît en `Ready`. Tous les pods système sont `Running` ou `Completed`.

Astuce

Le tunnel doit rester ouvert tant que vous utilisez `kubectl` ou `helm`. Laissez le terminal avec la commande `ssh -N` actif. Vous travaillez dans un **second terminal**. Créez un script `tunnel.sh` pour simplifier.

9.bis.3 — Script `tunnel.sh` (raccourci)

Hôte DevStack | `stack@devstack`

```
cat > ~/td2/ansible/tunnel.sh <<'EOF'
#!/bin/bash
FIP=$(cd ~/td2/terraform && terraform output -raw floating_ip)
pkill -f "ssh.*6443:127.0.0.1:6443" 2>/dev/null || true
sleep 1
sshpass -p azerty ssh -N \
  -o StrictHostKeyChecking=no \
  -o ExitOnForwardFailure=yes \
  -L 6443:127.0.0.1:6443 \
  ubuntu@"$FIP"
EOF
chmod +x ~/td2/ansible/tunnel.sh
```

Étape 10 — Chart Helm postgres

Helm — Déployer

Helm est le gestionnaire de paquets de Kubernetes. Il empaquette un ensemble de manifestes YAML dans un *chart*, versionné et paramétrable via `values.yaml`.

10.1 — Créer le namespace et le squelette

Hôte DevStack | `stack@devstack`

```
export KUBECONFIG=$HOME/td2/ansible/kubeconfig
kubectl create namespace td2

cd ~/td2/helm
helm create postgres
rm -rf postgres/templates/*
rm -f postgres/values.yaml
```

10.2 — `values.yaml`

Hôte DevStack | `stack@devstack`

```
cat > postgres/values.yaml <<'EOF'
image:
  repository: postgres
  tag: "16-alpine"

auth:
  username: app
  password: changeme
  database: appdb

```

```
storage:
  size: 1Gi
  storageClass: local-path

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 512Mi
EOF
```

10.3 — Service headless + ConfigMap init

Hôte DevStack | stack@devstack

```
cat > postgres/templates/service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
EOF

cat > postgres/templates/configmap-init.yaml <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-init
data:
  init.sql: |
    CREATE TABLE IF NOT EXISTS items (
      id SERIAL PRIMARY KEY,
      name TEXT NOT NULL
    );
    INSERT INTO items (name) VALUES
      ('item-alpha'), ('item-beta'), ('item-gamma')
    ON CONFLICT DO NOTHING;
EOF
```

10.4 — StatefulSet avec PGDATA

Piège classique

Sans PGDATA, Postgres peut crasher. Sur certains backends de stockage (notamment `local-path` en K3s), le point de montage contient un dossier `lost+found` qui fait échouer `initdb`. Solution : définir `PGDATA` pour utiliser un sous-dossier.

Hôte DevStack | stack@devstack

```
cat > postgres/templates/statefulset.yaml <<'EOF'
```

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              value: "{{ .Values.auth.username }}"
            - name: POSTGRES_PASSWORD
              value: "{{ .Values.auth.password }}"
            - name: POSTGRES_DB
              value: "{{ .Values.auth.database }}"
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
            - name: init
              mountPath: /docker-entrypoint-initdb.d
          resources:
{{ toYaml .Values.resources | indent 12 }}
      volumes:
        - name: init
          configMap:
            name: postgres-init
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: "{{ .Values.storage.storageClass }}"
        resources:
          requests:
            storage: "{{ .Values.storage.size }}"
EOF

helm lint postgres
helm install postgres ./postgres -n td2 --wait --timeout 3m
```

Critère de validation

Le pod `postgres-0` est `Running`. Le PVC `data-postgres-0` est `Bound`. Test SQL :

```
kubectl -n td2 exec postgres-0 -- psql -U app -d appdb -c "SELECT * FROM items;"
```

3 lignes attendues.

Étape 11 — Chart Helm backend

11.1 — Squelette et valeurs

Hôte DevStack | stack@devstack

```
cd ~/td2/helm
helm create backend
rm -rf backend/templates/*
rm -f backend/values.yaml

cat > backend/values.yaml <<'EOF'
replicaCount: 2

image:
  repository: python
  tag: "3.12-slim"

service:
  port: 5000

db:
  host: postgres.td2.svc.cluster.local
  port: 5432
  user: app
  password: changeme
  name: appdb

resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 300m
    memory: 256Mi
EOF
```

11.2 — ConfigMap code + Deployment + Service

Hôte DevStack | stack@devstack

```
cat > backend/templates/configmap-code.yaml <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-code
data:
  app.py: |
    import os, socket, json
    from http.server import BaseHTTPRequestHandler, HTTPServer
    import pg8000.native

    DB = dict(
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        host=os.environ["DB_HOST"],
```

```
        database=os.environ["DB_NAME"],
    )

    class Handler(BaseHTTPRequestHandler):
        def do_GET(self):
            if self.path.startswith("/api/items"):
                try:
                    conn = pg8000.native.Connection(**DB)
                    rows = conn.run("SELECT id, name FROM items ORDER BY id")
                    conn.close()
                    body = json.dumps({
                        "pod": socket.gethostname(),
                        "items": [{"id": r[0], "name": r[1]} for r in rows],
                    }).encode()
                    self.send_response(200)
                    self.send_header("Content-Type", "application/json")
                    self.end_headers()
                    self.wfile.write(body)
                except Exception as e:
                    self.send_response(500)
                    self.end_headers()
                    self.wfile.write(str(e).encode())
            else:
                self.send_response(404)
                self.end_headers()

        def log_message(self, *args):
            pass

    if __name__ == "__main__":
        HTTPServer(("0.0.0.0", 5000), Handler).serve_forever()
EOF

cat > backend/templates/deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      initContainers:
        - name: install-deps
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          command: ["sh", "-c"]
          args:
            - pip install --quiet --no-cache-dir --target /pip pg8000
          volumeMounts:
            - name: pip
              mountPath: /pip
      containers:
        - name: backend
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          command: ["python", "/app/app.py"]
```

```
        ports:
          - containerPort: 5000
        env:
          - name: PYTHONPATH
            value: /pip
          - name: DB_HOST
            value: "{{ .Values.db.host }}"
          - name: DB_PORT
            value: "{{ .Values.db.port }}"
          - name: DB_USER
            value: "{{ .Values.db.user }}"
          - name: DB_PASSWORD
            value: "{{ .Values.db.password }}"
          - name: DB_NAME
            value: "{{ .Values.db.name }}"
        volumeMounts:
          - name: code
            mountPath: /app
          - name: pip
            mountPath: /pip
        resources:
{{ toYaml .Values.resources | indent 12 }}
      volumes:
        - name: code
          configMap:
            name: backend-code
        - name: pip
          emptyDir: {}
EOF

cat > backend/templates/service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  selector:
    app: backend
  ports:
    - port: {{ .Values.service.port }}
      targetPort: 5000
EOF

helm lint backend
helm install backend ./backend -n td2 --wait --timeout 3m
```

11.3 — Test du backend

Piège classique

Erreur `Extra data` avec `kubectl run -rm`. `kubectl run -rm` ajoute une ligne « pod deleted » après la sortie du conteneur, ce qui casse `json.load`. Filtrez la sortie avec `grep`.

Hôte DevStack | `stack@devstack`

```
# Test simple : recuperer les items
kubectl -n td2 run test-curl --rm -i --restart=Never \
  --image=curlimages/curl -- \
  curl -s http://backend:5000/api/items
```

```
# Test du load-balancing (methode robuste)
for i in 1 2 3 4; do
  kubectl -n td2 run test-curl-$i --rm -i --restart=Never \
    --image=curlimages/curl -- \
    curl -s http://backend:5000/api/items 2>/dev/null \
    | tr -d '\r' | grep -oE '\{"pod":[^,]+' | head -1
done
```

Critère de validation

Les 2 noms de pods backend alternent. Le JSON contient les 3 items.

Étape 12 — Chart Helm `frontend`

12.1 — Squelette et valeurs

Hôte DevStack | `stack@devstack`

```
cd ~/td2/helm
helm create frontend
rm -rf frontend/templates/*
rm -f frontend/values.yaml

cat > frontend/values.yaml <<'EOF'
replicaCount: 2

image:
  repository: nginx
  tag: "alpine"

service:
  port: 80

ingress:
  enabled: true
  className: traefik

resources:
  requests:
    cpu: 20m
    memory: 32Mi
  limits:
    cpu: 100m
    memory: 128Mi
EOF
```

12.2 — ConfigMap nginx avec header `X-Pod-Name`

Astuce

Pourquoi le header `X-Pod-Name` ? Sans lui, impossible de prouver visuellement le load-balancing depuis `curl` : le HTML contient un placeholder rempli par JavaScript. Le header est exposé par nginx à chaque réponse, et lisible en `curl -I`.

Hôte DevStack | `stack@devstack`

```
cat > frontend/templates/configmap-nginx.yaml <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: frontend-nginx
data:
  default.conf: |
    server {
      listen 80;
      server_name _;

      # Expose le hostname du pod (preuve du load-balancing)
      add_header X-Pod-Name $hostname always;

      location / {
        root /usr/share/nginx/html;
        index index.html;
      }

      location /api/ {
        proxy_pass http://backend.td2.svc.cluster.local:5000/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
      }
    }
EOF
```

12.3 — Le HTML complet (interface NovaShop)

Rappel

Interface enrichie. On déploie un vrai frontend qui affiche :

— une card **bleue** = pod frontend (change à chaque refresh)
— une card **orange** = pod backend (change aussi)
— une card **verte** = PostgreSQL (stable)
— un tableau stylé avec les items issus de la DB
— un auto-refresh toutes les 5 s = preuve visuelle du round-robin

Hôte DevStack | `stack@devstack`

```
cat > frontend/templates/configmap-html.yaml <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: frontend-html
data:
  index.html: |
    <!doctype html>
    <html lang="fr">
    <head>
      <meta charset="utf-8">
      <title>NovaShop - TD2</title>
      <style>
        * { box-sizing: border-box; }
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-
            serif;
               margin: 0; padding: 2em; background: #f0f4f8; color: #1f2937; }
```

```
      .container { max-width: 900px; margin: 0 auto; }
      header { display: flex; align-items: center; justify-content: space-between;
               margin-bottom: 2em; }
      h1 { color: #1950a0; margin: 0; font-size: 2em; }
      .tagline { color: #6b7280; font-size: 0.9em; margin-top: 0.2em; }
      .cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr
          ));
               gap: 1em; margin-bottom: 2em; }
      .card { background: #fff; padding: 1em 1.2em; border-radius: 8px;
              box-shadow: 0 1px 3px rgba(0,0,0,0.08); border-left: 4px solid #1950a0; }
      .card.backend { border-left-color: #e6781e; }
      .card.db { border-left-color: #1e8c46; }
      .card-label { font-size: 0.75em; text-transform: uppercase; letter-spacing: 0.05
          em;
                    color: #6b7280; margin-bottom: 0.3em; }
      .card-value { font-family: ui-monospace, "SF Mono", Menlo, monospace;
                    font-size: 0.95em; color: #111827; word-break: break-all; }
      table { width: 100%; border-collapse: collapse; background: #fff;
              border-radius: 8px; overflow: hidden;
              box-shadow: 0 1px 3px rgba(0,0,0,0.08); }
      thead { background: #1950a0; color: #fff; }
      th { text-align: left; padding: 0.8em 1em; font-weight: 600; }
      td { padding: 0.7em 1em; border-bottom: 1px solid #e5e7eb; }
      tbody tr:last-child td { border-bottom: none; }
      tbody tr:hover { background: #f9fafb; }
      .status { display: inline-block; padding: 0.2em 0.6em; border-radius: 999px;
                background: #d1fae5; color: #065f46; font-size: 0.8em; font-weight: 600;
                    }
      .refresh-note { text-align: center; color: #9ca3af; font-size: 0.85em;
                      margin-top: 2em; }
      .error { background: #fee2e2; color: #991b1b; padding: 1em; border-radius: 8px;
               border-left: 4px solid #dc2626; }
    </style>
  </head>
  <body>
    <div class="container">
      <header>
        <div>
          <h1>NovaShop</h1>
          <div class="tagline">Application 3-tiers &mdash; TD2 IaC / DevSecOps</div>
        </div>
        <span class="status" id="status">&#9679; En ligne</span>
      </header>

      <div class="cards">
        <div class="card">
          <div class="card-label">Pod frontend</div>
          <div class="card-value" id="frontend-pod">&mdash;</div>
        </div>
        <div class="card backend">
          <div class="card-label">Pod backend</div>
          <div class="card-value" id="backend-pod">&mdash;</div>
        </div>
        <div class="card db">
          <div class="card-label">Base de donnees</div>
          <div class="card-value">PostgreSQL 16</div>
        </div>
      </div>

      <table>
        <thead><tr><th>ID</th><th>Nom de l'article</th></tr></thead>
```

```
        <tbody id="items-body">
          <tr><td colspan="2" style="text-align:center;color:#9ca3af;">Chargement...</td></tr>
        </tbody>
      </table>

      <div class="refresh-note">Rafra&icirc;chissement automatique toutes les 5 secondes</div>
    </div>

    <script>
      async function refresh() {
        try {
          const headResp = await fetch('/?cachebust=' + Date.now(), { method: 'HEAD' });
          const fpod = headResp.headers.get('X-Pod-Name') || '?';
          document.getElementById('frontend-pod').textContent = fpod;

          const apiResp = await fetch('/api/items?cachebust=' + Date.now());
          const data = await apiResp.json();
          document.getElementById('backend-pod').textContent = data.pod || '?';

          const tbody = document.getElementById('items-body');
          tbody.innerHTML = '';
          if (!data.items || data.items.length === 0) {
            tbody.innerHTML = '<tr><td colspan="2" style="text-align:center;color:#9ca3af;">Aucun article</td></tr>';
          } else {
            for (const it of data.items) {
              const tr = document.createElement('tr');
              tr.innerHTML = '<td>' + it.id + '</td><td>' + it.name + '</td>';
              tbody.appendChild(tr);
            }
          }
          document.getElementById('status').innerHTML = '&#9679; En ligne';
          document.getElementById('status').style.background = '#d1fae5';
          document.getElementById('status').style.color = '#065f46';
        } catch (e) {
          document.getElementById('status').innerHTML = '&#9679; Hors ligne';
          document.getElementById('status').style.background = '#fee2e2';
          document.getElementById('status').style.color = '#991b1b';
          document.getElementById('items-body').innerHTML =
            '<tr><td colspan="2" class="error">Erreur : ' + e + '</td></tr>';
        }
      }

      refresh();
      setInterval(refresh, 5000);
    </script>
  </body>
  </html>
EOF
```

12.4 — Deployment, Service, Ingress

Hôte DevStack | stack@devstack

```
cat > frontend/templates/deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: frontend
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: nginx
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 80
          volumeMounts:
            - name: html
              mountPath: /usr/share/nginx/html
            - name: nginx-conf
              mountPath: /etc/nginx/conf.d
          resources:
{{ toYaml .Values.resources | indent 12 }}
      volumes:
        - name: html
          configMap:
            name: frontend-html
        - name: nginx-conf
          configMap:
            name: frontend-nginx
EOF

cat > frontend/templates/service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  selector:
    app: frontend
  ports:
    - port: {{ .Values.service.port }}
      targetPort: 80
EOF

cat > frontend/templates/ingress.yaml <<'EOF'
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend
spec:
  ingressClassName: {{ .Values.ingress.className }}
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
```

```
              port:
                number: {{ .Values.service.port }}
{{- end }}
EOF

helm lint frontend
helm install frontend ./frontend -n td2 --wait --timeout 3m
```

Étape 13 — Validation bout en bout

Hôte DevStack | stack@devstack

```
cd ~/td2/terraform
export FIP=$(terraform output -raw floating_ip)
echo "URL : http://$FIP"

# 1) Releases Helm
helm list -n td2

# 2) Objets K8s
kubectl -n td2 get pods,svc,statefulset,ingress,pvc

# 3) Endpoints
kubectl -n td2 get endpoints

# 4) Test HTTP simple
curl -I "http://$FIP" | head -3

# 5) Load-balancing frontend (via header X-Pod-Name)
for i in $(seq 1 6); do
  curl -sI "http://$FIP" | grep -i '^X-Pod-Name'
done

# 6) Chaine complete /api/items
curl -s "http://$FIP/api/items" | python3 -m json.tool
```

Critère de validation

Les 6 sorties sont vertes :

1. 3 releases Helm en `deployed` ;
2. 5 pods `Running`, l'Ingress `frontend` présent ;
3. Endpoints de `frontend` et `backend` avec 2 IPs chacune ;
4. `HTTP/1.1 200 OK` ;
5. **Header `X-Pod-Name` qui alterne entre 2 valeurs** ;
6. JSON complet avec le pod backend + les 3 items.

13.1 — Preuve visuelle (navigateur)

Ouvrez `http://<FIP>` dans un navigateur. Vous devez voir :

— Cards **frontend** et **backend** qui affichent des noms de pods ;
— Ces noms qui **changent toutes les 5 s** (auto-refresh) ;
— Tableau des 3 items ;
— Badge « **En ligne** » vert.

Faites une capture d'écran pour le rendu.

Étape 14 — Idempotence et self-healing

14.1 — Test d'idempotence

Hôte DevStack | stack@devstack

```
# Terraform
cd ~/td2/terraform && terraform plan # No changes

# Ansible
cd ~/td2/ansible && ansible-playbook playbook.yml # changed=0

# Helm
cd ~/td2/helm
helm upgrade postgres ./postgres -n td2
helm upgrade backend ./backend -n td2
helm upgrade frontend ./frontend -n td2
kubectl -n td2 get pods # noms de pods inchanges
```

14.2 — Self-healing

Hôte DevStack | stack@devstack

```
kubectl -n td2 get pods -l app=frontend
POD=$(kubectl -n td2 get pods -l app=frontend -o name | head -1)
echo "Suppression de $POD"
kubectl -n td2 delete $POD
kubectl -n td2 get pods -l app=frontend -w
```

Réflexion d'architecte

En quelques secondes, un nouveau pod apparaît. Le Service continue de servir du trafic grâce à l'autre replica. C'est le **self-healing** : Kubernetes maintient en permanence l'état cible. Aucune intervention manuelle.

Étape 15 — Exercice bonus — Compteur de visites

Réflexion d'architecte

Cet exercice est facultatif, mais fortement recommandé. Il est conçu pour vous faire manipuler les **trois tiers** d'un coup (DB, backend, frontend) et vivre un **cycle de déploiement complet** : modification → `helm upgrade` → rollout → rollback. C'est exactement ce que vous ferez en production.
Faites cet exercice AVANT l'étape 16 (destruction). Sinon vous devrez tout redéployer pour le tester.

Contexte

NovaShop veut mesurer son audience. L'équipe produit demande : « **Afficher en temps réel le nombre total de visites de la page d'accueil.** »

Tier	Action
PostgreSQL	Créer une table `visits` (id, pod_name, created_at)
Backend	Ajouter un endpoint `GET /api/visits` qui enregistre une visite et renvoie le total
Frontend	Ajouter une card « Visites » qui appelle `/api/visits` à chaque refresh
Déploiement	`helm upgrade` sur `backend` et `frontend`
Validation	Rollout successful, compteur qui s'incrémente

Étape A — Migration SQL

Piège classique

Le `init.sql` du ConfigMap ne rejoue PAS au `helm upgrade`. Il n'est exécuté qu'à la **création initiale** du volume. Toute modification ultérieure doit être migrée *manuellement* (ou avec un outil dédié type Flyway).

Hôte DevStack | `stack@devstack`

```
export KUBECONFIG=$HOME/td2/ansible/kubeconfig

kubectl -n td2 exec postgres-0 -- psql -U app -d appdb -c "
CREATE TABLE IF NOT EXISTS visits (
  id SERIAL PRIMARY KEY,
  pod_name TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);
"

kubectl -n td2 exec postgres-0 -- psql -U app -d appdb -c "\d visits"
```

Critère de validation

La table `visits` apparaît avec 3 colonnes.

Étape B — Modifier le backend

Hôte DevStack | `stack@devstack`

```
cd ~/td2/helm

# Sauvegarder la version precedente
cp backend/templates/configmap-code.yaml backend/templates/configmap-code.yaml.bak

cat > backend/templates/configmap-code.yaml <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-code
data:
  app.py: |
    import os, socket, json
    from http.server import BaseHTTPRequestHandler, HTTPServer
    import pg8000.native

    DB = dict(
```

```
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        host=os.environ["DB_HOST"],
        database=os.environ["DB_NAME"],
    )

    class Handler(BaseHTTPRequestHandler):
        def do_GET(self):
            if self.path.startswith("/api/items"):
                try:
                    conn = pg8000.native.Connection(**DB)
                    rows = conn.run("SELECT id, name FROM items ORDER BY id")
                    conn.close()
                    body = json.dumps({
                        "pod": socket.gethostname(),
                        "items": [{"id": r[0], "name": r[1]} for r in rows],
                    }).encode()
                    self._send(200, body)
                except Exception as e:
                    self._send(500, str(e).encode())
            elif self.path.startswith("/api/visits"):
                try:
                    conn = pg8000.native.Connection(**DB)
                    conn.run("INSERT INTO visits (pod_name) VALUES (:p)",
                             p=socket.gethostname())
                    rows = conn.run("SELECT COUNT(*) FROM visits")
                    total = rows[0][0]
                    conn.close()
                    body = json.dumps({
                        "pod": socket.gethostname(),
                        "total_visits": total,
                    }).encode()
                    self._send(200, body)
                except Exception as e:
                    self._send(500, str(e).encode())
            else:
                self._send(404, b"")

        def _send(self, code, body):
            self.send_response(code)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)

        def log_message(self, *args):
            pass

    if __name__ == "__main__":
        HTTPServer(("0.0.0.0", 5000), Handler).serve_forever()
EOF
```

Étape C — Modifier le frontend

Hôte DevStack | stack@devstack

```
cd ~/td2/helm
cp frontend/templates/configmap-html.yaml frontend/templates/configmap-html.yaml.bak
nano frontend/templates/configmap-html.yaml
```

Modifications à apporter :

1. HTML — Ajouter cette card après celle de PostgreSQL :

```
<div class="card visits">
  <div class="card-label">Visites totales</div>
  <div class="card-value" id="visits">-</div>
</div>
```

2. CSS — Ajouter dans le `<style>` :

```
.card.visits { border-left-color: #6e3c96; }
```

3. JavaScript — Ajouter à la fin de la fonction `refresh()` :

```
const visitResp = await fetch('/api/visits?cachebust=' + Date.now());
const visitData = await visitResp.json();
document.getElementById('visits').textContent = visitData.total_visits;
```

Étape D — Déployer

Hôte DevStack | stack@devstack

```
cd ~/td2/helm

helm upgrade backend ./backend -n td2 --wait --timeout 3m
helm upgrade frontend ./frontend -n td2 --wait --timeout 3m

kubectl -n td2 rollout status deployment/backend
kubectl -n td2 rollout status deployment/frontend
kubectl -n td2 get pods
```

Définition

Rollout. Procédure *progressive* par laquelle Kubernetes remplace les anciens pods par les nouveaux. Par défaut (RollingUpdate), il démarre un nouveau pod, attend qu'il soit `Ready`, puis tue un ancien. **Zéro downtime** si le nombre de replicas est suffisant.

Critère de validation

`rollout status` affiche `successfully rolled out`. Les pods ont de nouveaux hash (ReplicaSet changé).

Étape E — Valider

Hôte DevStack | stack@devstack

```
export FIP=$(cd ~/td2/terraform && terraform output -raw floating_ip)

for i in $(seq 1 5); do
  curl -s "http://$FIP/api/visits"
  echo ""
done

kubectl -n td2 exec postgres-0 -- psql -U app -d appdb -c \
  "SELECT COUNT(*), COUNT(DISTINCT pod_name) FROM visits;"
```

Critère de validation

Le `total_visits` s'incrémente à chaque appel. 5 lignes en base, 2 pods distincts. La card violette « Visites » augmente toutes les 5 s.

Étape F — Rollback (bonus)

Hôte DevStack | `stack@devstack`

```
helm history backend -n td2
helm rollback backend -n td2
kubectl -n td2 rollout status deployment/backend

# Verifier : /api/visits doit redevenir 404
curl -sI "http://$FIP/api/visits" | head -1
```

Critère de validation

`helm history` montre 3 révisions. Après rollback, `/api/visits` renvoie 404.

Réflexion d'architecte

Question de synthèse. En production, comment garantir qu'une migration de base de données s'exécute **avant** que le nouveau backend ne démarre ? Rédigez 3 lignes.
Indice : `initContainers` et **Helm hooks** (`pre-upgrade`).

Rendu de l'exercice

— **Capture d'écran** de l'interface avec la card « Visites » incrémentée.
— Sortie de `helm history backend -n td2` (3 révisions).
— Sortie de `kubectl -n td2 rollout status deployment/backend`.
— Réponse à la question de synthèse (3 lignes).
— **Q bonus** — Pourquoi le `init.sql` du ConfigMap ne rejoue-t-il pas au `helm upgrade` ? (2 lignes)

Point DevSecOps

Attention au sens de migration. Sur votre environnement de test, la migration manuelle est tolérée. En production, une migration *ratée* peut corrompre la base. Les outils dédiés (Flyway, Liquibase, Atlas) historisent chaque migration, garantissent l'ordre et l'atomicité, et permettent un rollback.

Étape 16 — Destruction propre

Piège classique

Faites vos captures d'écran AVANT de lancer la destruction. Une fois supprimée, l'infrastructure ne peut plus être restaurée (il faudrait tout recréer).

16.1 — Désinstaller Helm

Hôte DevStack | `stack@devstack`

```
helm uninstall frontend -n td2
helm uninstall backend -n td2
helm uninstall postgres -n td2

# Helm ne supprime pas les PVC par defaut
kubectl -n td2 delete pvc --all
kubectl delete namespace td2
```

16.2 — Détruire l'infrastructure OpenStack

Hôte DevStack | `stack@devstack`

```
cd ~/td2/terraform
terraform destroy
# yes
```

Critère de validation

`Destroy complete! Resources: 7 destroyed.` Les ressources partagées (`asl-net`, `asl-key`, image, projet) **restent**.

Point DevSecOps

Terraform ne touche qu'à ce qu'il possède. Les `data sources` ne sont jamais détruites. C'est la base d'un travail d'équipe sain sur une infra partagée : un `destroy` d'applicatif ne casse pas le socle réseau de toute l'entreprise.

16.3 — Nettoyage local

Hôte DevStack | `stack@devstack`

```
pkill -f "ssh.*6443:127.0.0.1:6443" || true # fermer le tunnel
unset KUBECONFIG
rm -f ~/td2/ansible/kubeconfig ~/td2/ansible/inventory.ini
```

Diagnostic guidé

Symptôme	Outil	Cause fréquente	Commande
`No valid host`	Terraform	Ressources insuffisantes	`openstack hypervisor stats show`
`No image found`	Terraform	Nom d'image invalide	`openstack image list`
Warning `compute_floatingip`	Terraform	Utilisation de l'ancienne ressource	voir Erratum 1
`UNREACHABLE` Ansible	Ansible	Mauvais mot de passe	`ansible -m ping all`
DNS ou MTU	Ansible	systemd-resolved	`cat /etc/resolv.conf`
`Connection refused` 6443	kubectl	Tunnel SSH absent	`ss -tlnp | grep :6443`
`Extra data` JSON	kubectl	Sortie `kubectl run` polluée	voir Erratum 4
Postgres `CrashLoop`	Helm	`PGDATA` manquant	voir Erratum 2
Pas de `X-Pod-Name`	Helm	ConfigMap nginx ancien	voir Erratum 3
`pip install` lent	Helm	Réseau / image	`kubectl logs`
Ingress 404	Helm	Service mal nommé	`kubectl describe ingress`

Rendu attendu

Déposez un **PDF de 3 pages maximum** + un **dépôt Git** contenant :

1. L'arborescence du projet : `terraform/`, `ansible/`, `helm/`.
2. Un schéma de l'architecture (Terraform → Ansible → Helm).
3. La sortie de `terraform output`.
4. La sortie du `PLAY RECAP` Ansible (les 2 passages, dont `changed=0`).
5. La sortie de `helm list -n td2`.
6. La sortie de `kubectl -n td2 get pods,svc,ingress`.
7. La preuve du load-balancing frontend (header `X-Pod-Name` alternant).
8. La sortie de `curl -s http://<FIP>/api/items` avec le JSON.
9. **Une capture d'écran de l'interface web** `NovaShop`.
10. **Q1** — Pourquoi utilise-t-on des `data sources` pour `asl-net` et pas des `resources` ? Que se passerait-il si on inversait ?
11. **Q2** — Quelle est la frontière exacte entre ce que fait Terraform et ce que fait Ansible ? Donnez un exemple de tâche qui ne peut pas aller dans l'autre outil.
12. **Q3** — Pourquoi un `StatefulSet` et pas un `Deployment` pour PostgreSQL ? Citez 2 différences concrètes.
13. **Q4** — Expliquez en 3 lignes pourquoi on utilise un **tunnel SSH** plutôt que d'ouvrir le port 6443 dans le SG.
14. **Q5** — Après `terraform destroy`, quelles ressources OpenStack subsistent ? Pourquoi est-ce volontaire ?

Nettoyage obligatoire

Voir l'étape 15. En résumé :

Hôte DevStack | `stack@devstack`

```
# Helm
helm uninstall frontend backend postgres -n td2
kubectl delete pvc --all -n td2
kubectl delete namespace td2

# Terraform
cd ~/td2/terraform && terraform destroy

# Local
pkill -f "ssh.*6443:127.0.0.1:6443" || true
rm -f ~/td2/ansible/kubeconfig ~/td2/ansible/inventory.ini
```

Point DevSecOps

FinOps & hygiène. Une VM oubliée = facture, surface d'attaque, ressources saturées pour les camarades. En entreprise, la destruction reproductible (`terraform destroy` dans un pipeline) est un **objectif de sécurité**.

Fin du TD2 v2 — prochaine étape : TD3, cluster multi-nœuds et industrialisation avancée (registry privé, Ansible Vault, Sealed Secrets).